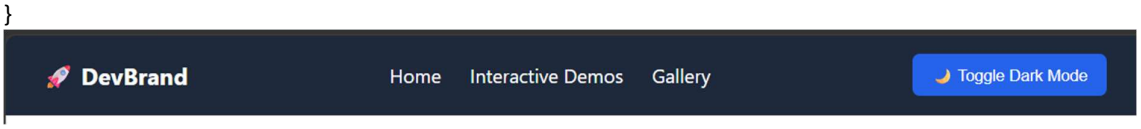
Unit II Assignment
Full Stack
// Khushi Choudhary
// CU240251564

Question 1: CSS Flexbox Navbar
HTML Structure

```
<nav class="navbar">
 <div class="logo">BrandLogo</div>
 <ul class="nav-links">
  <li><a href="#home">Home</a></li>
  <li><a href="#about">About</a></li>
  <li><a href="#contact">Contact</a></li>
 </ul>
</nav>
```

CSS Flexbox Code

```
.navbar {
 display: flex;
 justify-content: space-between;
 align-items: center;
 padding: 1rem 2rem;
 background-color: #1e293b;
 color: #ffffff;
}
.nav-links {
 display: flex;
 gap: 1.5rem;
 list-style: none;
 margin: 0;
 padding: 0;
}
.nav-links a {
 color: #ffffff;
 text-decoration: none;
}
```



Question 2: Event Listener Bug Fix (Bug Identification)
The method name addEventListner contains a typo (missing the e before ner). The correct standard DOM method name is addEventListener.
Corrected Code

```
const button = document.querySelector('.toggle-btn');
button.addEventListener('click', () => {
 document.body.classList.toggle('dark-mode');
});
```

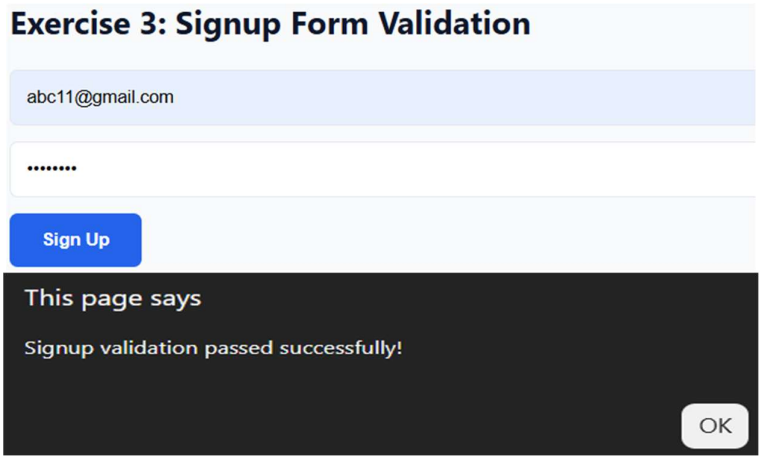
Question 3: Vanilla JavaScript Signup Form Validation
HTML Form

```
<form id="signupForm">
 <input type="email" id="email" placeholder="Email" required />
 <input type="password" id="password" placeholder="Password" required />
```

```
 <button type="submit">Sign Up</button>
 <p id="error-message" style="color: red;"></p>
</form>
```

Vanilla JavaScript Validation

```
const form = document.getElementById('signupForm');
const emailInput = document.getElementById('email');
const passwordInput = document.getElementById('password');
const errorMsg = document.getElementById('error-message');
form.addEventListener('submit', (e) => {
 const email = emailInput.value.trim();
 const password = passwordInput.value;
 if (!email.includes('@')) {
  e.preventDefault();
  errorMsg.textContent = 'Please enter a valid email containing "@".';
  return;
 }
 if (password.length < 8) {
  e.preventDefault();
  errorMsg.textContent = 'Password must be at least 8 characters long.';
  return;
 }
 errorMsg.textContent = '';
});
```



Question 4: Memory Leak Fix in Event Listener

Memory Leak Explanation:

Each invocation of renderPage() attaches an anonymous event listener callback function that stays permanently registered on the global window object.

As renderPage() executes repeatedly, duplicate callback instances accumulate in memory without garbage collection, compounding CPU work on every scroll event.

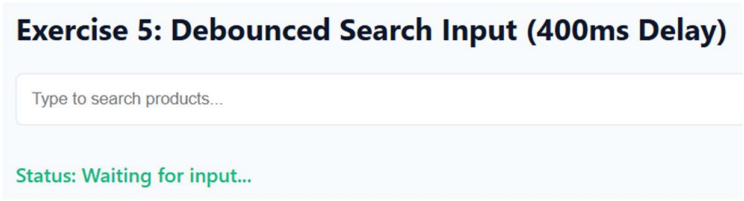
Rewritten Code

```
// Extract named listener callback function
function handleScroll() {
 console.log(window.scrollY);
}
function renderPage() {
 // Remove existing listener before re-adding to prevent duplicates
 window.removeEventListener('scroll', handleScroll);
 window.addEventListener('scroll', handleScroll);
```

```
}
renderPage();
renderPage();
```

Question 5: Debounced Search-Input Handler (JavaScript Code)

```
function debounce(func, delay = 400) {
  let timerId;
  return function (...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => {
      func.apply(this, args);
    }, delay);
  };
}
// API Function
function searchProducts(query) {
  console.log(` Executing API search for query: "${query}"` );
}
// Debounced input handler
const searchInput = document.getElementById('searchInput');
const handleSearch = debounce((event) => {
  const query = event.target.value.trim();
  if (query) {
    searchProducts(query);
  }
}, 400);
searchInput.addEventListener('input', handleSearch);
```



Question 6: Mobile Media Query Layout Fix

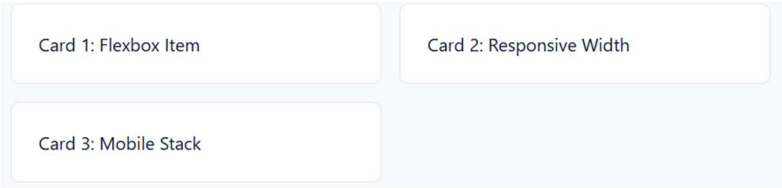
Why Original CSS Breaks:

The fixed width: 960px on .container exceeds small mobile screen widths (e.g., 480px), causing horizontal scrollbar overflow and content clipping.

Fixed CSS with Media Query

```
.container {
  display: flex;
  width: 100%;
  max-width: 960px;
  flex-wrap: wrap;
}
.card {
  width: 300px;
}
/* Mobile Media Query for screens below 480px */
@media (max-width: 480px) {
  .container {
    flex-direction: column;
    width: 100%;
```

```
 }
 .card {
  width: 100%;
 }
}
```



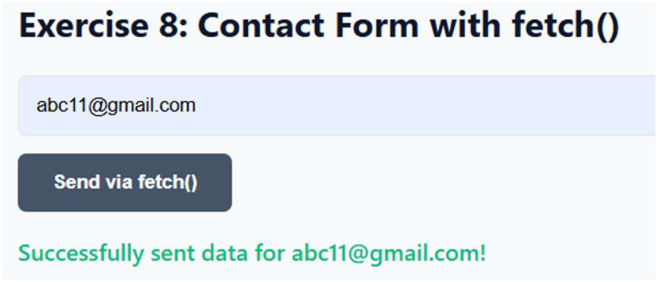
Question 7: Persistent Theme withlocalStorage (JavaScript Code)

```
// Function to apply theme and persist choice in localStorage
function setTheme(theme) {
 document.body.className = theme;
 localStorage.setItem('user-theme', theme);
}
// Restore saved theme on page load
function loadSavedTheme() {
 const savedTheme = localStorage.getItem('user-theme') || 'light';
 document.body.className = savedTheme;
}
// Restore on page load
document.addEventListener('DOMContentLoaded', loadSavedTheme);
// Toggle button listener
const themeToggleBtn = document.getElementById('themeToggleBtn');
themeToggleBtn.addEventListener('click', () => {
 const currentTheme = document.body.classList.contains('dark') ? 'dark' : 'light';
 const newTheme = currentTheme === 'dark' ? 'light' : 'dark';
 setTheme(newTheme);
});
```

Question 8: AJAX Form Submission withfetch() (Rewritten Code (e.preventDefault() + fetch))

```
const contactForm = document.getElementById('contactForm');
contactForm.addEventListener('submit', async (e) => {
 e.preventDefault(); // Prevents full page refresh and state loss
 const formData = new FormData(contactForm);
 const data = Object.fromEntries(formData.entries());
 try {
  const response = await fetch('/api/contact', {
   method: 'POST',
   headers: {
    'Content-Type': 'application/json'
   },
   body: JSON.stringify(data)
  });
  if (response.ok) {
   const result = await response.json();
   console.log('Submitted successfully:', result);
  }
 } catch (error) {
  console.error('Fetch submission error:', error);
```

```
  }
});
```



Question 9: Missingawaitin Async Function

Missing Keyword Identificatio:

The function is missing the await keyword before fetch(...) and res.json(). Both fetch() and .json() return Promises that must be awaited to resolve response data.

Corrected Code

```
async function getUser(id) {
  const res = await fetch(` /api/users/${id}` );
  const data = await res.json();
  return data;
}
```

Question 10: CSS Grid Gallery (minmax+auto-fit) HTML

```
<div class="image-gallery">
  <img src="image1.jpg" alt="Gallery Image 1" />
  <img src="image2.jpg" alt="Gallery Image 2" />
  <img src="image3.jpg" alt="Gallery Image 3" />
  <img src="image4.jpg" alt="Gallery Image 4" />
</div>
```

CSS Grid Code

```
.image-gallery {
  display: grid;
  /* Automatically wraps from 4 columns on desktop (~200px+) to 2 columns on mobile (~180px) */
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1rem;
}
.image-gallery img {
  width: 100%;
  height: 200px;
  object-fit: cover;
  border-radius: 8px;
}
```

